FAST — National University of Computer and Emerging Sciences Karachi Campus

Department of Software Engineering

PF POSTLAB TASKS

Course Code: C1002

Subject: Programming Fundamentals

Section: D & C

Instructor: Mr. Shayan Ali

STUDENT SUBMISSION DETAILS

Name: AMNA KHAN

Roll No / ID: 26K-3091

Class / Section: C

Date of Submission: 28-OCTOBER'2026

Part A: Three-Dimensional Arrays

Task 1 (Student Result Analysis):

A university has 2 sections, each containing 3 students. Eachstudent has marks in 4 subjects. Store the marks in a 3D array and write a C program to display the total and average marks of each student, find the highest-scoring student in each section, count students whose average is at least 50, and identify the section with the highest overall average.

(globals)

Untitled1.cpp

```c
#include<stdio.h>

int main()
{
    int marks[2][3][4];
    int i, j, k;
    int total, count = 0;
    int highestStudent, highestTotal;
    int sectionTotal[2] = {0, 0};
    float avg, sectionAvg[2];

    // Input marks
    for(i = 0; i < 2; i++)
    {
        printf("\nSection %d:\n", i + 1);

        for(j = 0; j < 3; j++)
        {
            printf("Enter marks of Student %d:\n", j + 1);

            for(k = 0; k < 4; k++)
            {
                printf("Subject %d: ", k + 1);
                scanf("%d", &marks[i][j][k]);
            }
        }
    }

    // total and average of each student
    for(i = 0; i < 2; i++)
    {
        printf("\n--- Section %d Results ---\n", i + 1);
```

Untitled1.cpp

```c
    }

    // total and average of each student
    for(i = 0; i < 2; i++)
    {
        printf("\n--- Section %d Results ---\n", i + 1);

        highestTotal = -1;
        highestStudent = 0;

        for(j = 0; j < 3; j++)
        {
            total = 0;

            for(k = 0; k < 4; k++)
            {
                total = total + marks[i][j][k];
            }

            avg = total / 4.0;
            sectionTotal[i] = sectionTotal[i] + total;

            printf("Student %d Total = %d\n", j + 1, total);
            printf("Student %d Average = %.2f\n", j + 1, avg);

            if(avg >= 50)
            {
                count++;
            }

            if(total > highestTotal)
            {
                highestTotal = total;
```

Untitled1.cpp

```c
            if(total > highestTotal)
            {
                highestTotal = total;
                highestStudent = j;
            }
        }

        printf("Highest scoring student: Student %d\n",highestStudent + 1);
        printf("Highest total marks: %d\n", highestTotal);

        sectionAvg[i] = sectionTotal[i] / 12.0;
    }

    //students with average at least 50
    printf("\nStudents with average at least 50: %d\n", count);

    // section averages
    printf("Section 1 Overall Average = %.2f\n", sectionAvg[0]);
    printf("Section 2 Overall Average = %.2f\n", sectionAvg[1]);

    if(sectionAvg[0] > sectionAvg[1])
    {
        printf("Section 1 has the highest overall average.\n");
    }
    else if(sectionAvg[1] > sectionAvg[0])
    {
        printf("Section 2 has the highest overall average.\n");
    }
    else
    {
        printf("Both sections have equal overall averages.\n");
```

Untitled1.cpp

```
            highestStudent = j;
        }
    }

    printf("Highest scoring student: Student %d\n",highestStudent + 1);
    printf("Highest total marks: %d\n", highestTotal);

    sectionAvg[i] = sectionTotal[i] / 12.0;
}

//students with average at least 50
printf("\nStudents with average at least 50: %d\n", count);

// section averages
printf("Section 1 Overall Average = %.2f\n", sectionAvg[0]);
printf("Section 2 Overall Average = %.2f\n", sectionAvg[1]);

if(sectionAvg[0] > sectionAvg[1])
{
    printf("Section 1 has the highest overall average.\n");
}
else if(sectionAvg[1] > sectionAvg[0])
{
    printf("Section 2 has the highest overall average.\n");
}
else
{
    printf("Both sections have equal overall averages.\n");
}

return 0;
}
```

```
Section 1:
Enter marks of Student 1:
Subject 1: 20
Subject 2: 30
Subject 3: 40
Subject 4: 50
Enter marks of Student 2:
Subject 1: 50
Subject 2: 60
Subject 3: 70
Subject 4: 80
Enter marks of Student 3:
Subject 1: 90
Subject 2: 80
Subject 3: 85
Subject 4: 79

Section 2:
Enter marks of Student 1:
Subject 1: 40
Subject 2: 50
Subject 3: 30
Subject 4: 20
Enter marks of Student 2:
Subject 1: 10
Subject 2: 40
Subject 3: 30
Subject 4: 50
Enter marks of Student 3:
Subject 1: 50
Subject 2: 60
Subject 3: 80
```

```
Subject 4: 50
Enter marks of Student 3:
Subject 1: 50
Subject 2: 60
Subject 3: 80
Subject 4: 70

--- Section 1 Results ---
Student 1 Total = 140
Student 1 Average = 35.00
Student 2 Total = 260
Student 2 Average = 65.00
Student 3 Total = 334
Student 3 Average = 83.50
Highest scoring student: Student 3
Highest total marks: 334

--- Section 2 Results ---
Student 1 Total = 140
Student 1 Average = 35.00
Student 2 Total = 130
Student 2 Average = 32.50
Student 3 Total = 260
Student 3 Average = 65.00
Highest scoring student: Student 3
Highest total marks: 260

Students with average at least 50: 3
Section 1 Overall Average = 61.17
Section 2 Overall Average = 44.17
Section 1 has the highest overall average.

--------------------------------
```

Task 2 (Monthly Sales Analysis):

A company has 2 branches, sells 3 products, and records sales quantities for 4 weeks. Store the sales data in a 3D array and write a C program to calculate the total quantity sold for each product in each branch, identify the product with the highest sales in each branch, find the week with the highest overall sales, and determine which branch sold the greatest quantity of products.

*] Untitled1.cpp

```c
#include <stdio.h>

int main()
{
    int sales[2][3][4];
    int i, j, k;
    int total, branchTotal;
    int maxProduct, maxSales;
    int weekTotal, maxWeek, highestWeek;
    int branch1 = 0, branch2 = 0;

    // Input sales data
    for(i = 0; i < 2; i++)
    {
        printf("\nEnter sales for Branch %d:\n", i + 1);

        for(j = 0; j < 3; j++)
        {
            printf("Product %d:\n", j + 1);

            for(k = 0; k < 4; k++)
            {
                printf("Week %d: ", k + 1);
                scanf("%d", &sales[i][j][k]);
            }
        }
    }

    // Total sales of each product in each branch
    for(i = 0; i < 2; i++)
    {
        printf("\nBranch %d:\n", i + 1);
```

[*] Untitled1.cpp

```c
    // Total sales of each product in each branch
    for(i = 0; i < 2; i++)
    {
        printf("\nBranch %d:\n", i + 1);

        maxSales = -1;

        for(j = 0; j < 3; j++)
        {
            total = 0;

            for(k = 0; k < 4; k++)
            {
                total = total + sales[i][j][k];
            }

            printf("Product %d total = %d\n", j + 1, total);

            if(total > maxSales)
            {
                maxSales = total;
                maxProduct = j + 1;
            }

            if(i == 0)
                branch1 = branch1 + total;
            else
                branch2 = branch2 + total;
        }

        printf("Highest-selling product: Product %d\n",
               maxProduct);
```

[*] Untitled1.cpp

```
        }

        printf("Highest-selling product: Product %d\n",
               maxProduct);
    }

    // week with highest overall sales
    maxWeek = -1;

    for(k = 0; k < 4; k++)
    {
        weekTotal = 0;

        for(i = 0; i < 2; i++)
        {
            for(j = 0; j < 3; j++)
            {
                weekTotal = weekTotal + sales[i][j][k];
            }
        }

        printf("Week %d total sales = %d\n", k + 1, weekTotal);

        if(weekTotal > maxWeek)
        {
            maxWeek = weekTotal;
            highestWeek = k + 1;
        }
    }

    printf("\nHighest overall sales week: Week %d\n", highestWeek);

    // Compare branches
```

[*] Untitled1.cpp

```
        for(i = 0; i < 2; i++)
        {
            for(j = 0; j < 3; j++)
            {
                weekTotal = weekTotal + sales[i][j][k];
            }
        }

        printf("Week %d total sales = %d\n", k + 1, weekTotal);

        if(weekTotal > maxWeek)
        {
            maxWeek = weekTotal;
            highestWeek = k + 1;
        }
    }

    printf("\nHighest overall sales week: Week %d\n", highestWeek);

    // Compare branches
    printf("Branch 1 total sales = %d\n", branch1);
    printf("Branch 2 total sales = %d\n", branch2);

    if(branch1 > branch2)
        printf("Branch 1 sold the greatest quantity.\n");
    else if(branch2 > branch1)
        printf("Branch 2 sold the greatest quantity.\n");
    else
        printf("Both branches sold equal quantities.\n");

    return 0;
}
```

```
Enter sales for Branch 1:
Product 1:
Week 1: 2
Week 2: 4
Week 3: 5
Week 4: 6
Product 2:
Week 1: 2
Week 2: 7
Week 3: 5
Week 4: 3
Product 3:
Week 1: 7
Week 2: 8
Week 3: 5
Week 4: 6

Enter sales for Branch 2:
Product 1:
Week 1: 3
Week 2: 5
Week 3: 7
Week 4: 9
Product 2:
Week 1: 4
Week 2: 8
Week 3: 6
Week 4: 5
Product 3:
Week 1: 6
Week 2: 2
Week 3: 9
Week 4: 5
```

```
Week 2: 8
Week 3: 6
Week 4: 5
Product 3:
Week 1: 6
Week 2: 2
Week 3: 9
Week 4: 5

Branch 1:
Product 1 total = 17
Product 2 total = 17
Product 3 total = 26
Highest-selling product: Product 3

Branch 2:
Product 1 total = 24
Product 2 total = 23
Product 3 total = 22
Highest-selling product: Product 1
Week 1 total sales = 24
Week 2 total sales = 34
Week 3 total sales = 37
Week 4 total sales = 34

Highest overall sales week: Week 3
Branch 1 total sales = 60
Branch 2 total sales = 69
Branch 2 sold the greatest quantity.

--------------------------------
Process exited after 41.99 seconds with return value 0
Press any key to continue . . .
```

Task 3 (Hospital Temperature Monitoring):

A hospital monitors 2 wards, each containing 3patients. The temperature of each patient is recorded 4 times. Store the readings in a 3D array and write a C program to calculate each patient's average temperature, count readings above 37.5°C, find the highest temperature recorded in each ward, and identify patients whose average temperature exceeds 38°C.

[*] Untitled1.cpp

```c
#include <stdio.h>

int main()
{
    float temp[2][3][4];
    float sum, avg, highest;
    int i, j, k;
    int count = 0;

    // Input temperature readings
    for(i = 0; i < 2; i++)
    {
        printf("\nEnter temperatures for Ward %d:\n", i + 1);

        for(j = 0; j < 3; j++)
        {
            printf("Patient %d:\n", j + 1);

            for(k = 0; k < 4; k++)
            {
                printf("Reading %d: ", k + 1);
                scanf("%f", &temp[i][j][k]);
            }
        }
    }

    // average temperature of each patient
    for(i = 0; i < 2; i++)
    {
        printf("\nWard %d:\n", i + 1);

        highest = temp[i][0][0];
```

[*] Untitled1.cpp

```
26
27      // average temperature of each patient
28      for(i = 0; i < 2; i++)
29      {
30          printf("\nWard %d:\n", i + 1);
31
32          highest = temp[i][0][0];
33
34          for(j = 0; j < 3; j++)
35          {
36              sum = 0;
37
38              for(k = 0; k < 4; k++)
39              {
40                  sum = sum + temp[i][j][k];
41
42                  if(temp[i][j][k] > 37.5)
43                  {
44                      count++;
45                  }
46
47                  if(temp[i][j][k] > highest)
48                  {
49                      highest = temp[i][j][k];
50                  }
51              }
52
53              avg = sum / 4;
54
55              printf("Patient %d average = %.2f C\n",j + 1, avg);
56
57              if(avg > 38.0)
58              {
```

[*] Untitled1.cpp

```
38              for(k = 0; k < 4; k++)
39              {
40                  sum = sum + temp[i][j][k];
41
42                  if(temp[i][j][k] > 37.5)
43                  {
44                      count++;
45                  }
46
47                  if(temp[i][j][k] > highest)
48                  {
49                      highest = temp[i][j][k];
50                  }
51              }
52
53              avg = sum / 4;
54
55              printf("Patient %d average = %.2f C\n",j + 1, avg);
56
57              if(avg > 38.0)
58              {
59                  printf("Patient %d has average above 38 C\n",j + 1);
60              }
61          }
62
63          printf("Highest temperature in Ward %d = %.2f C\n",i + 1, highest);
64      }
65
66      printf("\nTotal readings above 37.5 C = %d\n", count);
67
68      return 0;
69  }
```

```
Enter temperatures for Ward 1:
Patient 1:
Reading 1: 33
Reading 2: 36
Reading 3: 5
Reading 4: 37
Patient 2:
Reading 1: 38
Reading 2: 39
Reading 3: 36
Reading 4: 7
Patient 3:
Reading 1: 33
Reading 2: 34
Reading 3: 37
Reading 4: 38

Enter temperatures for Ward 2:
Patient 1:
Reading 1: 44
Reading 2: 46
Reading 3: 47
Reading 4: 45
Patient 2:
Reading 1: 39
Reading 2: 36
Reading 3: 35
Reading 4: 38
Patient 3:
Reading 1: 33
Reading 2: 36
Reading 3: 35
```

```
Reading 4: 38

Enter temperatures for Ward 2:
Patient 1:
Reading 1: 44
Reading 2: 46
Reading 3: 47
Reading 4: 45
Patient 2:
Reading 1: 39
Reading 2: 36
Reading 3: 35
Reading 4: 38
Patient 3:
Reading 1: 33
Reading 2: 36
Reading 3: 35
Reading 4: 38

Ward 1:
Patient 1 average = 27.75 C
Patient 2 average = 30.00 C
Patient 3 average = 35.50 C
Highest temperature in Ward 1 = 39.00 C

Ward 2:
Patient 1 average = 45.50 C
Patient 1 has average above 38 C
Patient 2 average = 37.00 C
Patient 3 average = 35.50 C
Highest temperature in Ward 2 = 47.00 C

Total readings above 37.5 C = 10

--------------------------------
Process exited after 48.73 seconds with return value 0
```

Part B: User-Defined Functions (UDFs)

Task 4 (Electricity Bill Calculator):

Write a C program that calculates an electricity bill based on units consumed and customer category. Domestic customers are charged Rs. 15 per unit for the first 100 units and Rs. 20 per unit for additional units, while commercial customers are charged Rs. 25 per unit for all units consumed. If the bill exceeds Rs. 5,000, apply a 5% surcharge. Display the units consumed, bill amount, surcharge, and final payable amount.

[*] Untitled1.cpp

```c
#include <stdio.h>

float calculateBill(int units, int category)
{
    float bill;

    if(category == 1)
    {
        if(units <= 100)
            bill = units * 15;
        else
            bill = (100 * 15) + ((units - 100) * 20);
    }
    else
    {
        bill = units * 25;
    }

    return bill;
}

int main()
{
    int units, category;
    float bill, surcharge, finalBill;

    printf("Enter units consumed: ");
    scanf("%d", &units);

    printf("Enter customer category:\n");
    printf("1. Domestic\n");
    printf("2. Commercial\n");
    scanf("%d", &category);
```

```c
[*] Untitled1.cpp
    int units, category;
    float bill, surcharge, finalBill;

    printf("Enter units consumed: ");
    scanf("%d", &units);

    printf("Enter customer category:\n");
    printf("1. Domestic\n");
    printf("2. Commercial\n");
    scanf("%d", &category);

    if(units < 0 || (category != 1 && category != 2))
    {
        printf("Invalid input!\n");
        return 0;
    }

    bill = calculateBill(units, category);

    if(bill > 5000)
        surcharge = bill * 0.05;
    else
        surcharge = 0;

    finalBill = bill + surcharge;

    printf("\nUnits consumed = %d\n", units);
    printf("Bill amount = Rs. %.2f\n", bill);
    printf("Surcharge = Rs. %.2f\n", surcharge);
    printf("Final payable amount = Rs. %.2f\n", finalBill);

    return 0;
}
```

```
C:\Users\humai\Desktop\PF T
Enter units consumed: 400
Enter customer category:
1. Domestic
2. Commercial
1

Units consumed = 400
Bill amount = Rs. 7500.00
Surcharge = Rs. 375.00
Final payable amount = Rs. 7875.00

--------------------------------
Process exited after 10.32 seconds with return value 0
Press any key to continue . . .
```

Task 5 (Employee Salary and Bonus):

Write a C program that calculates an employee's final salary based on basic salary and years of service. Employees with more than 5 years of service receive a 15% bonus, those with 3–5 years receive a 10% bonus, and those with less than 3 years receive a 5% bonus. If the salary after adding the bonus exceeds Rs. 100,000, deduct 5% tax. Display the basic salary, bonus, tax, and final salary.

[*] Untitled1.cpp

```c
#include <stdio.h>

float calculateBonus(float salary, int years)
{
    float bonus;

    if(years > 5)
        bonus = salary * 0.15;
    else if(years >= 3)
        bonus = salary * 0.10;
    else
        bonus = salary * 0.05;

    return bonus;
}

int main()
{
    float salary, bonus, tax, finalSalary;
    int years;

    printf("Enter basic salary: Rs. ");
    scanf("%f", &salary);

    printf("Enter years of service: ");
    scanf("%d", &years);

    if(salary < 0 || years < 0)
    {
        printf("Invalid input!\n");
        return 0;
    }
```

[*] Untitled1.cpp

```c
    int years;

    printf("Enter basic salary: Rs. ");
    scanf("%f", &salary);

    printf("Enter years of service: ");
    scanf("%d", &years);

    if(salary < 0 || years < 0)
    {
        printf("Invalid input!\n");
        return 0;
    }

    bonus = calculateBonus(salary, years);

    finalSalary = salary + bonus;

    if(finalSalary > 100000)
        tax = finalSalary * 0.05;
    else
        tax = 0;

    finalSalary = finalSalary - tax;

    printf("\nBasic Salary = Rs. %.2f\n", salary);
    printf("Bonus = Rs. %.2f\n", bonus);
    printf("Tax = Rs. %.2f\n", tax);
    printf("Final Salary = Rs. %.2f\n", finalSalary);

    return 0;
}
```

```
Enter basic salary: Rs. 80000
Enter years of service: 3

Basic Salary = Rs. 80000.00
Bonus = Rs. 8000.00
Tax = Rs. 0.00
Final Salary = Rs. 88000.00

--------------------------------
Process exited after 15.16 seconds with return value 0
Press any key to continue . . .
```

Task 6 (Loan Eligibility Checker):

Write a C program that determines whether a person qualifies for a loan based on age, monthly salary, and existing monthly loan payments. The applicant must be at least 21 years old, earn a monthly salary of at least Rs. 50,000, and have at least 40% of their salary remaining after existing payments. If eligible, calculate the maximum loan amount as 10 times the remaining monthly income. Otherwise, display the reason for ineligibility.

(globals)

[*] Untitled1.cpp

```c
#include <stdio.h>

float remainingIncome(float salary, float payments)
{
    return salary - payments;
}

int main()
{
    int age;
    float salary, payments, remaining, maxLoan;

    printf("Enter age: ");
    scanf("%d", &age);

    printf("Enter monthly salary: Rs. ");
    scanf("%f", &salary);

    printf("Enter existing monthly loan payments: Rs. ");
    scanf("%f", &payments);

    if(age < 0 || salary < 0 || payments < 0)
    {
        printf("Invalid input!\n");
        return 0;
    }

    remaining = remainingIncome(salary, payments);

    if(age < 21)
    {
        printf("Not eligible: Age must be at least 21.\n");
    }
```

[*] Untitled1.cpp

```c
    if(age < 0 || salary < 0 || payments < 0)
    {
        printf("Invalid input!\n");
        return 0;
    }

    remaining = remainingIncome(salary, payments);

    if(age < 21)
    {
        printf("Not eligible: Age must be at least 21.\n");
    }
    else if(salary < 50000)
    {
        printf("Not eligible: Salary must be at least Rs. 50,000.\n");
    }
    else if(remaining < salary * 0.40)
    {
        printf("Not eligible: At least 40%% of salary must remain.\n");
    }
    else
    {
        maxLoan = remaining * 10;

        printf("\nLoan Eligible!\n");
        printf("Remaining monthly income = Rs. %.2f\n", remaining);
        printf("Maximum loan amount = Rs. %.2f\n", maxLoan);
    }

    return 0;
}
```

```
Enter age: 50
Enter monthly salary: Rs. 1100000
Enter existing monthly loan payments: Rs. 40000

Loan Eligible!
Remaining monthly income = Rs. 1060000.00
Maximum loan amount = Rs. 10600000.00

--------------------------------
Process exited after 20.1 seconds with return value 0
Press any key to continue . . .
```

Task 7 (Number Property Analyzer):

Write a C program that accepts a positive integer and determines whether it is even or odd, whether it is a palindrome, and whether it is a prime number. A palindrome number remains the same when its digits are reversed. Use arithmetic operations and loops to perform the checks without converting the number into a string or using arrays. Display all results.

Untitled1.cpp

```c
#include <stdio.h>

int main()
{
    int n, original, remainder, reverse = 0;
    int i, isPrime = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    if(n <= 0)
    {
        printf("Please enter a positive integer.\n");
        return 0;
    }

    original = n;

    // even or odd
    if(n % 2 == 0)
        printf("The number is Even.\n");
    else
        printf("The number is Odd.\n");

    // palindrome
    while(n > 0)
    {
        remainder = n % 10;
        reverse = reverse * 10 + remainder;
        n = n / 10;
    }
```

```
Untitled1.cpp
29          reverse = reverse * 10 + remainder;
30          n = n / 10;
31      }
32
33      if(original == reverse)
34          printf("The number is a Palindrome.\n");
35      else
36          printf("The number is not a Palindrome.\n");
37
38      // prime number
39      if(original == 1)
40          isPrime = 0;
41
42      for(i = 2; i <= original / 2; i++)
43      {
44          if(original % i == 0)
45          {
46              isPrime = 0;
47              break;
48          }
49      }
50
51      if(isPrime == 1)
52          printf("The number is Prime.\n");
53      else
54          printf("The number is not Prime.\n");
55
56      return 0;
57  }
58
```

```
C:\Users\humai\Desktop\PF T
Enter a positive integer: 11
The number is Odd.
The number is a Palindrome.
The number is Prime.

--------------------------------
Process exited after 4.139 seconds with return value 0
Press any key to continue . . .
```

Task 8 (Delivery Charges Calculator):

Write a C program that calculates delivery charges based on parcel weight, delivery distance, and customer membership. Parcels weighing up to 2 kg cost Rs. 200, those weighing above 2 kg and up to 5 kg cost Rs. 400, and those weighing above 5 kg cost Rs. 700. Add Rs. 20 per kilometre for distances exceeding 10 km. Members receive a 10% discount if the total charges exceed Rs. 1,000. Display the base charges, distance charges, discount, and final amount.

[*] Untitled1.cpp

```c
#include <stdio.h>

float calculateBase(float weight)
{
    if(weight <= 2)
        return 200;
    else if(weight <= 5)
        return 400;
    else
        return 700;
}

int main()
{
    float weight, distance;
    float base, distanceCharges, total, discount, finalAmount;
    int member;

    printf("Enter parcel weight in kg: ");
    scanf("%f", &weight);

    printf("Enter delivery distance in km: ");
    scanf("%f", &distance);

    printf("Are you a member? (1 = Yes, 0 = No): ");
    scanf("%d", &member);

    if(weight <= 0 || distance < 0 ||
       (member != 0 && member != 1))
    {
```

[*] Untitled1.cpp

```c
       (member != 0 && member != 1))
    {
        printf("Invalid input!\n");
        return 0;
    }

    base = calculateBase(weight);

    if(distance > 10)
        distanceCharges = (distance - 10) * 20;
    else
        distanceCharges = 0;

    total = base + distanceCharges;

    if(member == 1 && total > 1000)
        discount = total * 0.10;
    else
        discount = 0;

    finalAmount = total - discount;

    printf("\nBase charges = Rs. %.2f\n", base);
    printf("Distance charges = Rs. %.2f\n", distanceCharges);
    printf("Discount = Rs. %.2f\n", discount);
    printf("Final amount = Rs. %.2f\n", finalAmount);

    return 0;
}
```

```
Enter parcel weight in kg: 40
Enter delivery distance in km: 50
Are you a member? (1 = Yes, 0 = No): 1

Base charges = Rs. 700.00
Distance charges = Rs. 800.00
Discount = Rs. 150.00
Final amount = Rs. 1350.00

--------------------------------
Process exited after 16.71 seconds with return value 0
Press any key to continue . . .
```

Task 9 (ATM Transaction System):

Write a menu-driven C program that starts with a balance of Rs. 20,000 and allows the user to check the balance, deposit money, withdraw money, or exit. Deposits and withdrawals must be positive, and withdrawals cannot exceed the available balance or Rs. 10,000 per transaction. If the balance falls below Rs. 5,000 after a withdrawal, display a low-balance warning. Continue displaying the menu until the user selects Exit, and display the updated balance after every successful transaction.

[*] Untitled1.cpp

```c
#include <stdio.h>

int main()
{
    int choice;
    float balance = 20000, amount;

    do
    {
        printf("\n===== ATM MENU =====\n");
        printf("1. Check Balance\n");
        printf("2. Deposit Money\n");
        printf("3. Withdraw Money\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Current Balance = Rs. %.2f\n", balance);
                break;

            case 2:
                printf("Enter deposit amount: Rs. ");
                scanf("%f", &amount);

                if(amount > 0)
                {
                    balance = balance + amount;
```

[*] Untitled1.cpp

```c
                    balance = balance + amount;
                    printf("Deposit successful!\n");
                    printf("Updated Balance = Rs. %.2f\n", balance);
                }
                else
                {
                    printf("Invalid amount! Deposit must be positive.\n");
                }
                break;

            case 3:
                printf("Enter withdrawal amount: Rs. ");
                scanf("%f", &amount);

                if(amount <= 0)
                {
                    printf("Invalid amount! Withdrawal must be positive.\n");
                }
                else if(amount > 10000)
                {
                    printf("Withdrawal cannot exceed Rs. 10,000.\n");
                }
                else if(amount > balance)
                {
                    printf("Insufficient balance!\n");
                }
                else
                {
                    balance = balance - amount;
```

```
*] Untitled1.cpp
53                  {
54                      printf("Insufficient balance!\n");
55                  }
56                  else
57                  {
58                      balance = balance - amount;
59
60                      printf("Withdrawal successful!\n");
61                      printf("Updated Balance = Rs. %.2f\n", balance);
62
63                      if(balance < 5000)
64                      {
65                          printf("Warning: Low balance!\n");
66                      }
67                  }
68                  break;
69
70              case 4:
71                  printf("Thank you for using the ATM!\n");
72                  break;
73
74              default:
75                  printf("Invalid choice! Please try again.\n");
76          }
77
78      } while(choice != 4);
79
80      return 0;
81  }
82
```

```
===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 1
Current Balance = Rs. 20000.00

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 2
Enter deposit amount: Rs. 1000
Deposit successful!
Updated Balance = Rs. 21000.00

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 3
Enter withdrawal amount: Rs. 2000
Withdrawal successful!
Updated Balance = Rs. 19000.00

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 4
Thank you for using the ATM!

--------------------------------
Process exited after 33.7 seconds with return value 0
Press any key to continue . . .
```

Task 10 (Shopping Bill Calculator):

Write a C program that calculates a shopping bill using product price, quantity, and customer membership status. If the subtotal exceeds Rs. 5,000, apply a 10% discount. Members receive an additional 5% discount on the subtotal. After applying the discounts, add a 2% payment fee if the customer pays by card. Display the subtotal, each applicable discount, payment fee, and final payable amount.

[*] Untitled1.cpp

```c
#include <stdio.h>

float calculateSubtotal(float price, int quantity)
{
    return price * quantity;
}

int main()
{
    float price, subtotal, discount1, discount2;
    float fee, finalBill;
    int quantity, member, payment;

    printf("Enter product price: Rs. ");
    scanf("%f", &price);

    printf("Enter quantity: ");
    scanf("%d", &quantity);

    printf("Are you a member? (1 = Yes, 0 = No): ");
    scanf("%d", &member);

    printf("Payment method (1 = Cash, 2 = Card): ");
    scanf("%d", &payment);

    if(price < 0 || quantity <= 0 ||(member != 0 && member != 1) ||(payment != 1 && payment != 2))
    {
        printf("Invalid input!\n");
        return 0;
    }
```

(globals)

[*] Untitled1.cpp

```c
        printf("Invalid input!\n");
        return 0;
    }

    subtotal = calculateSubtotal(price, quantity);

    // Discount for subtotal above Rs. 5000
    if(subtotal > 5000)
        discount1 = subtotal * 0.10;
    else
        discount1 = 0;

    // Additional discount for members
    if(member == 1)
        discount2 = subtotal * 0.05;
    else
        discount2 = 0;

    finalBill = subtotal - discount1 - discount2;

    // Add card payment fee
    if(payment == 2)
        fee = finalBill * 0.02;
    else
        fee = 0;

    finalBill = finalBill + fee;

    printf("\nSubtotal = Rs. %.2f\n", subtotal);
    printf("Discount (10%%) = Rs. %.2f\n", discount1);
    printf("Member discount (5%%) = Rs. %.2f\n", discount2);
    printf("Payment fee = Rs. %.2f\n", fee);
    printf("Final payable amount = Rs. %.2f\n", finalBill);

    return 0;
}
```

C:\Users\humai\Desktop\PF T

```
Enter product price: Rs. 2000
Enter quantity: 2
Are you a member? (1 = Yes, 0 = No): 1
Payment method (1 = Cash, 2 = Card): 2

Subtotal = Rs. 4000.00
Discount (10%) = Rs. 0.00
Member discount (5%) = Rs. 200.00
Payment fee = Rs. 76.00
Final payable amount = Rs. 3876.00

--------------------------------
Process exited after 11.95 seconds with return value 0
Press any key to continue . . .
```